

1 Description of Software Architecture

Our AUV's entire software stack runs on **ROS 2 (Humble)**. In this upgraded release, the architecture has evolved from an early prototype into a modular, production-grade ecosystem. We partitioned the codebase into discrete, decoupled packages – `auv_vision`, `auv_planner`, `auv_controls`, `auv_propulsion`, `auv_sensors`, and `auv_telemetry` – so that high-rate closed-loop propulsion, computer vision inference, deterministic state-machine planning, and black-box telemetry logging execute independently without blocking one another. All inter-node communication occurs strictly via asynchronous ROS 2 topics and standard message interfaces.

The complete software repository is structured within a clean workspace layout shown below:

```
auv_ws/src/  
|-- auv_bringup/  
|   |-- launch/ (sim_mission.launch.py, bridge.launch.py)  
|   '-- config/ (sim_params.yaml, real_params.yaml)  
|-- auv_vision/  
|   |-- gate_detector_node.py  
|   |-- gate_localizer_node.py  
|   |-- blue_bin_detector_node.py  
|   |-- blue_bin_cv_detector_node.py  
|   |-- blue_bin_localizer_node.py  
|   |-- green_detector_node.py  
|   |-- green_localizer_node.py  
|   |-- red_bin_detector_node.py  
|   |-- red_bin_localizer_node.py  
|   |-- qr_detector_node.py  
|   |-- qr_localizer_node.py  
|   '-- underwater_view_node.py  
|-- auv_planner/  
|   |-- gate_navigator_v2_node.py  
|   '-- green_navigator_node.py  
|-- auv_controls/  
|   '-- velocity_controller_node.py  
|-- auv_propulsion/  
|   '-- thruster_allocator_node.py  
|-- auv_sensors/  
|   |-- launch/ (state_estimation.launch.py)  
|   '-- config/ (ekf.yaml)  
|-- auv_telemetry/  
|   |-- mission_logger_node.py  
|   |-- mission_monitor_node.py  
|   '-- trail_mapper_node.py
```

Fig 1. High-Level Software Architecture & ROS 2 Package Organization

Rather than reviewing folders in isolation, the specific responsibilities, node architectures, and topic flows of each package are detailed organically alongside the respective vehicle functionalities in the sections that follow.

2 Salient Features of the Software Stack

- **Closed-Loop Physics Simulation & 8-Thruster Allocation** (`auv_controls` & `auv_propulsion`): Rather than perfectly tracking velocity commands via kinematic teleportation, the AUV is driven by 8 simulated physical thrusters. A closed-loop P-controller continuously calculates restorative body wrenches, which are translated into individual thruster forces via a Moore-Penrose pseudo-inverse allocation matrix.
- **Stereo-Vision Based 3D Localization** (`auv_vision`): Instead of relying on acoustic sensors, we compute 3D gate/bin coordinates by correlating YOLO bounding boxes with stereo disparity maps. The localizer interrogates four prioritized geometric candidate points across a 9×9 disparity patch, taking the median of valid pixels to apply the standard $\text{depth} = \text{fx} * \text{baseline} / \text{disparity}$ formula with a plausibility guard (0.2m–19.1m).
- **Dual-Camera Cross-Handoff**: The front stereo camera pair handles long-range acquisition and approach. The downward-facing camera takes over once the AUV is directly above a target, using odometry-based altitude for pinhole projection instead of stereo disparity. The system transitions seamlessly between them based on target coverage.
- **Dead-Reckoning Visual Fallback** (`auv_planner`): When targets exit the camera field-of-view during terminal approach, the navigators switch smoothly from live YOLO tracking to odometry dead-reckoning using the last verified target coordinates in the global odom frame, maintaining trajectory stability.
- **HSV-Based False-Positive Reduction**: For the bottom camera's blue-bin task, we built `blue_bin_cv_detector_node` which only accepts a blue blob if at least 15% of its pixel area spatially overlaps the green mat in the same frame. Open

blue water structurally cannot pass this check, eliminating false positives without needing additional neural network training data.

- **Active Safety Envelope & Wall Repulsion:** The controller actively damps roll and pitch oscillations via restoring torque if tilt exceeds 35°, enforces a safe depth envelope (−8.0m to +3.0m), and applies repulsive potential forces (300 N/m) when within 1.0m of any pool boundary.
- **Post-Mission Trail Mapping & Live Dashboard (`auv_telemetry`):** The `trail_mapper_node` records the AUV's trajectory every 0.1m, renders it as a colour-coded mission-progress plot with true-to-scale gate/bin/mat footprints on the pool boundary, and exports interactive 3D HTML maps, while `mission_monitor_node` provides a real-time ANSI terminal dashboard.

3 Logical Steps in the Working of Software Components

Mission planning is managed inside the `auv_planner` package by two coordinating finite state machine nodes: `gate_navigator_v2_node` for initial arena traversal and `green_navigator_node` for post-gate target centering.

3.1 Gate Navigation (`gate_navigator_v2_node`)

The gate navigator is a deliberately simplified *one-pass* FSM – it crosses the gate once and hands off immediately, with no U-turn or return leg of its own (that return trip is now part of the post-gate mission instead; see Step 5 in Section 3.2).

STEP 1: FORWARD DRIVE – When `sim_mission.launch.py` starts, the gate navigator begins in the FORWARD state. It drives straight ahead at 0.20 m/s for 3.5 metres using pure odometry. If the YOLO detector spots the gate during this phase, it skips directly to TRACK.

STEP 2: LATERAL SEARCH – If the gate was not acquired during the forward phase, the AUV enters SEARCH. It strafes sideways at 0.15 m/s, sweeping ± 6.0 metres from its center position while holding heading with a P-controller until a confident gate detection is registered.

STEP 3: TRACK AND APPROACH – In TRACK, the AUV drives toward a staging point 1.5m in front of the gate at up to 0.15 m/s while correcting its heading using the bearing error to the gate's odom-frame coordinates. If the YOLO feed goes stale (older than 1.5 seconds), it falls back to odom dead-reckoning to the last known position.

STEP 4: COMMIT, ALIGN, AND CROSS – Once within 0.5m of the staging point, the AUV locks its approach heading and enters ALIGN. It rotates in place until the yaw error drops below 0.05 rad (2.86°) for 8 consecutive ticks, then enters CROSS – driving straight through the gate with a 3.25m overshoot to clear the structure entirely, then publishes state "DONE" and hands off directly to `green_navigator_node`.

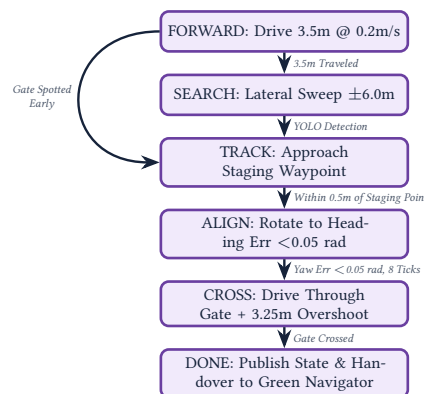


Fig 2. Gate Navigator State Machine (`auv_planner/gate_navigator_v2_node`) – one-pass only, no return leg.

3.2 Post-Gate Navigation (`green_navigator_node`)

After the gate navigator publishes "DONE", the `green_navigator_node` takes over through an automated multi-stage sequence covering the green mat, blue bin, payload drop, and – as of this revision – the full trip back to the start point:

1. **CLEAR_GATE:** Drives straight forward 0.46m (about half the AUV's length) immediately upon handover, opening clearance from the gate posts before any yaw turn.
2. **RESET_YAW:** Snaps heading back to the locked reference yaw published on `/auv/forward_reference_yaw` (the heading at mission start) – used again before the bin search so every lateral sweep below scans facing dead-ahead.
3. **SEARCH_GREEN:** Advance-and-sweep search for the green mat: strafes ± 20.0 m at 0.25 m/s, left first, creeping 3.0m

forward per full pass up to a 15.0m total advance budget. If the mat is never found, skips straight to RISE_FOR_SURVEY rather than stalling, since blue-bin YOLO detection is reliable on its own.

4. **APPROACH_GREEN:** Drives toward the green mat's odom coordinate until bottom -camera coverage, a front-camera on-mat sighting, or 2.0m proximity confirms it is over the zone.
5. **RISE_FOR_SURVEY:** Climbs exactly one robot-height (0.58m, taken from the AUV's own collision geometry) above its green-zone cruising depth, then RESET_YAW again before the bin search.
6. **SEARCH_BLUE_BIN:** Tighter sweep ($\pm 3.0\text{m}$, 8.0m advance budget) for the blue bin. If this budget is exhausted, the FSM falls back to an identical SEARCH_RED_BIN $\rightarrow$ APPROACH_RED_BIN $\rightarrow$ RISE_FOR_VERIFICATION_RED $\rightarrow$ CENTER_RED_BIN sequence for the red bin instead, tagging the eventual HOLD target as "red" rather than "blue".
7. **APPROACH_BLUE_BIN & RISE_FOR_VERIFICATION:** Drives to the stored bin coordinate, tapering speed on approach, then rises $\sim 0.5\text{m}$ for a wider bottom-camera view once within 0.35m or directly overhead.
8. **CENTER_BLUE_BIN & HOLD:** Bottom-camera visual servoing fine-centers the drum, requiring the error to stay under 0.35m for 10 consecutive ticks with the green mat confirmed underneath; the vehicle then holds position for 5 seconds.

New in This Revision

STEP 5: PAYLOAD DROP & RETURN LEG – Reaching the blue bin's HOLD state no longer ends the mission: the payload is released there, and the AUV then retraces its own path all the way back to the start point.

- **DESCEND_TO_BIN** – eases down half a robot-height below the HOLD altitude (only when the held target is the *blue* bin – a red-bin-only HOLD skips this and the drop, going straight to SURFACE_AND_TURN, since the payload only ever releases on the blue bin).
- **DROP_PAYLOAD** – holds position for 3 seconds while the payload releases.
- **SURFACE_AND_TURN** – builds the return route from the AUV's own recorded trail, reversed and downsampled to $\sim 0.5\text{m}$ spacing with the lateral search sweeps excluded (so the outbound zig-zag isn't replayed), then climbs back to the HOLD altitude and turns to face the first return waypoint.
- **RETRACE_PATH** – flies the reversed waypoint sequence, holding the same altitude the outbound trail recorded at each point (rather than a fixed cruise height) so it clears the gate structure on the way back, then publishes state "FINAL_DONE" once the start zone is reached.

This return leg replaces what earlier revisions of this document described as a gate-level U-turn: the AUV now retraces its own dead-reckoned trail across the *entire* mission (gate, green zone, and bin) rather than re-detecting and re-crossing just the gate.

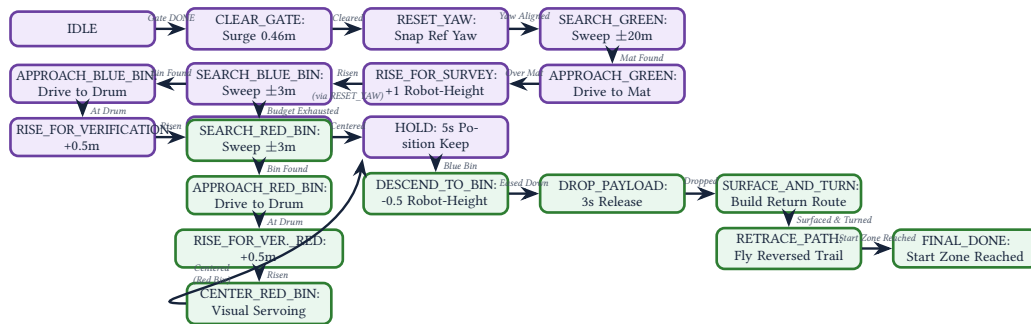


Fig 3. Post-Gate Navigator State Machine (auv_planner/green_navigator_node) – extended with the red-bin fallback and the payload-drop/return-leg states (highlighted).

4 Dataset Preparation and Object Classes

Building a reliable underwater object detection system meant we needed training data that accurately matched what our cameras see. Off-the-shelf datasets lack the competition arena's custom PVC gate dimensions, colored drums, and floor target mats, requiring a tailored dataset synthesis.

4.1 Data Sources

Simulation-Generated Images: We set up the full competition arena in Gazebo Harmonic using SDF world models (PVC gate, 8m×2m green target mat, blue drum, 3 red drums, colored flares, and QR-coded boxes). To capture comprehensive viewpoints, we developed an automated test harness (`bottom_cam_altitude_test.py`) that systematically teleports the AUV

across a coordinate grid over arena elements at 5 altitudes and 6 lateral offsets, capturing thousands of annotated training frames across varied viewing angles and distances.

Kaggle and External Sources: To reduce simulation overfitting, we supplemented our captures with underwater imagery from Kaggle datasets. These introduced natural variations in water turbidity, suspended particle backscatter, and dynamic lighting conditions.

4.2 Object Classes in the Dataset

Class	Description & Arena Physical Geometry	Detection & Localization Modality
Gate	Two vertical PVC posts with a connecting crossbar; 1.5m passage width	YOLOv8n + 4-Candidate Stereo Disparity
Blue Bin	Blue cylindrical drum (0.3m radius, 0.5m height) positioned on green mat	YOLOv8n (Front) + HSV Green-Overlap (Bottom)
Red Bin	Red cylindrical drums (same dimensions as blue); 3 placed on target mat	YOLOv8n + Altitude Pinhole Projection
QR Code	0.5m $\times$ 0.5m target cube with encoded mission payload markers on pool floor	OpenCV QRCodeDetector + Altitude Projection
Flares	Colored cylinders (orange, blue, red, yellow) placed at mid-water depth	YOLOv8n Bounding Box Inference
Green Target Mat	8m $\times$ 2m flat mat on pool floor; acts as landing zone for the bin task	Classical HSV Color Segmentation

4.3 Model Details

We use YOLOv8n (nano) as our detection backbone, providing the ideal balance of inference latency (45 FPS onboard) and mAP. For the green target mat, we skip neural inference entirely and use classical HSV color thresholding with morphological cleanup, which is lighter, faster, and immune to neural false positives. For the bottom camera's blue bin task, `blue_bin_cv_detector_node` verifies spatial overlap with the green mat, eliminating background blue-water triggers.

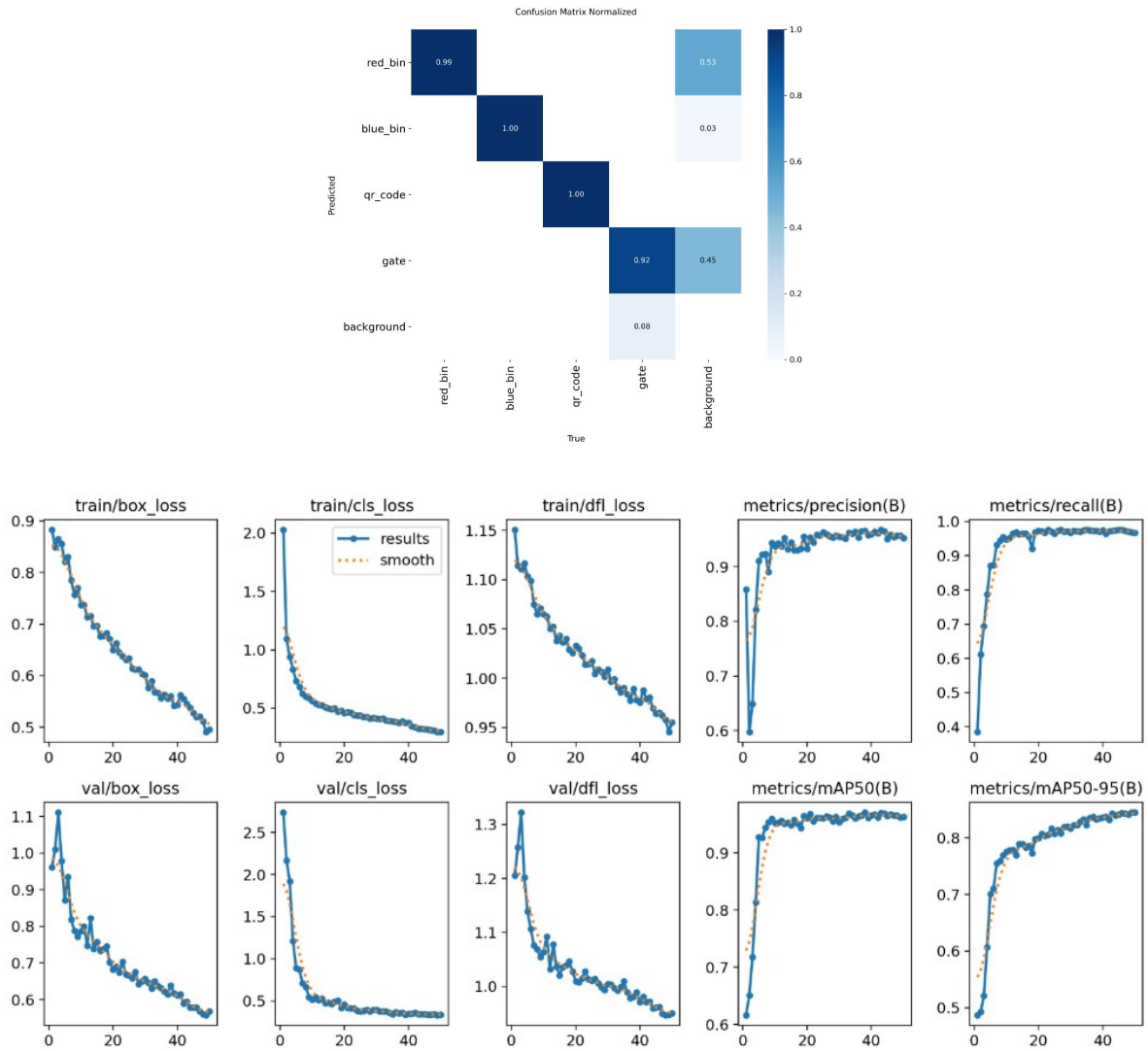


Fig 4/5. YOLOv8n Training Confusion Matrix (Normalized Across Arena Classes) and Training Performance Metrics (Loss Curves, Precision-Recall & mAP@0.5 Convergence).

5 Object Detection and Vision Pipeline

All perception nodes reside within the `auv_vision` package. The vision architecture divides perception into front stereo long-range estimation and downward monocular precision tracking:

5.1 Gate Detection Pipeline

The `gate_detector_node` subscribes to the left camera feed (`/model/auv_box/stereo_front/left/image_raw`). Frames are converted via CvBridge to OpenCV BGR arrays and processed through YOLOv8n (confidence threshold 0.10). The highest-confidence bounding box is published as a `Float32MultiArray` to `/auv/gate_detection_2d`.

The `gate_localizer_node` fuses this 2D detection with disparity maps from `stereo_image_proc`. Because bounding-box centers on a gate frame represent empty water, the localizer samples four geometric candidate points in priority order: Left Pole → Right Pole → Crossbar → Center. At each point, it interrogates a 9×9 disparity patch, accepts the first location with ≥ 3 valid pixels, computes the median disparity d , and calculates 3D coordinates:

$$Z_{depth} = (f_x \cdot B) / d, \quad X = (u - c_x) \cdot Z_{depth} / f_x, \quad Y = (v - c_y) \cdot Z_{depth} / f_y \quad (B = 0.075\text{m})$$

The resulting metric body position $[x, y, z, \text{conf}]$ is published to `/auv/gate_position_3d`.

5.2 Blue Bin Detection Pipeline

On the front camera, `blue_bin_detector_node` executes YOLOv8n inference. On the downward camera, `blue_bin_cv_detector_node` provides a robust classical fallback. It creates independent HSV green and blue masks, isolates blue contours, and calculates the intersection between each blue contour and the dilated green mat mask. A detection is only published to

`/auv/blue_bin_detection_bottom_2d` if at least 15% of the blue blob area overlaps the green mat, guaranteeing that open water never triggers a false positive.

5.3 Green Mat Detection Pipeline

The `green_detector_node` applies HSV segmentation ($H:38-82$, $S:80-255$, $V:60-255$) with morphological opening and closing. On the front camera, processing is confined to the lower 40% region of interest (ROI) because the floor mat only appears in the bottom portion of the horizontal field-of-view, discarding surface reflections. The downward camera evaluates the full frame to report coverage area and metric offset.

5.4 Red Bin, QR Code & Underwater View Rendering

The `red_bin_localizer_node` and `qr_localizer_node` project downward camera pixel coordinates into metric arena coordinates using odometric altitude: $X = (u - c_x) \cdot h/f_x$, $Y = (v - c_y) \cdot h/f_y$. The `qr_detector_node` decodes floor QR markers using OpenCV's `QRCodeDetector` and publishes strings to `/auv/qr_payload`. In parallel, `underwater_view_node` simulates Beer-Lambert optical attenuation ($I_\lambda = I_0 e^{-\alpha_\lambda z}$) on an isolated output topic for live visualization.

6 Controls, Propulsion & Thruster Allocation

Vehicle movement and hydrodynamic stabilization are managed collaboratively by `auv_controls` and `auv_propulsion`, bridging high-level planner velocity commands to physical thrusters:

6.1 8-Thruster Overactuated Geometry

The AUV features 8 brushless thrusters configured for full 6-DOF overactuated control. Thrusters $T_1 \dots T_4$ are mounted horizontally at $\pm 45^\circ$ angles relative to the hull's longitudinal axis, providing decoupled surge (F_x), sway (F_y), and yaw (M_z). Thrusters $T_5 \dots T_8$ are positioned vertically at the hull corners, providing heave (F_z), roll (M_x), and pitch (M_y).

6.2 Thruster Allocation Matrix (TAM) & Pseudo-Inverse

The relationship between desired body wrench $\tau = [F_x, F_y, F_z, M_x, M_y, M_z]^T$ and individual thruster thrusts $\mathbf{u} = [u_1, \dots, u_8]^T$ is governed by the thruster geometry matrix $\tau = \mathbf{B}\mathbf{u}$. The `thruster_allocator_node` solves for the optimal, minimum-energy thrust allocation using the Moore-Penrose pseudo-inverse:

$$\mathbf{u} = \mathbf{B}^T (\mathbf{B}\mathbf{B}^T)^{-1} \tau \Rightarrow \mathbf{u}_{horiz} = \mathbf{B}_H^+ [F_x, F_y, M_z]^T, \quad \mathbf{u}_{vert} = \mathbf{B}_V^+ [F_z, M_x, M_y]^T$$

Individual forces are clamped to $[-40\text{N}, +40\text{N}]$ and published to `/model/auv_box/joint/T_{1..8}/cmd_thrust`.

6.3 Closed-Loop Velocity Controller & Pose Differentiator

The `velocity_controller_node` consumes body velocity commands (`/model/auv_box/cmd_vel`) from navigators and compares them against vehicle velocities. To overcome Gazebo bridge limitations where twist messages in body frames are unpopulated, the node computes actual velocity via backward finite-differencing on odometry positions, smoothed by an Exponential Moving Average (EMA) filter ($\alpha = 0.3$):

$$v_{raw}[k] = (p[k] - p[k-1])/dt, \quad v_{filt}[k] = \alpha v_{raw}[k] + (1 - \alpha) v_{filt}[k-1]$$

Proportional control loops generate the required body wrench: $F_x = K_{p,x}(v_{x,cmd} - v_{x,filt})$, $F_y = K_{p,y}(v_{y,cmd} - v_{y,filt})$, and $M_z = K_{p,yaw}(\omega_{z,cmd} - \omega_{z,filt})$, published to `/auv/wrench_cmd`.

6.4 Triple-Layer Active Safety Envelope

- **Tipping Damping:** If roll ($|\phi|$) or pitch ($|\theta|$) exceeds 35° , the controller damps horizontal forces and commands restorative moments ($M_x = -35\phi$, $M_y = -35\theta$) to prevent capsizing.
- **Depth Envelope:** Enforces a hard operational depth envelope between -8.0m and $+3.0\text{m}$.
- **Pool Wall Repulsion:** When the vehicle approaches within 1.0m of any pool perimeter boundary, a continuous repulsive force $F_{rep} = 300 \times (1.0 - d_{wall})$ N pushes the hull away from the wall.

7 Sensor Suite, State Estimation & Sim-to-Real Portability

Sensors and filtering are managed inside `auv_sensors` and `auv_bringup`, providing clean, drift-free state estimates:

7.1 Sensor Suite & Camera Optical Specifications

Sensor Hardware	Topic / Interface	Rate / Resolution	Functional Role
Stereo Front Pair	.../stereo_front/...	30 Hz, Baseline 75mm	Forward obstacle detection & metric 3D disparity estimation.
Downward Camera	.../bottom/...	30 Hz, Wide-FOV HD	Target mat tracking, bin visual servoing, and QR code reading.
9-DOF IMU	/model/auv_box/imu	100 Hz, Triaxial	Measures angular rates and orientation for attitude hold.
Depth Sensor	/auv/odom (Z-Channel)	50 Hz, 1mm Res	Hydrostatic depth measurement for vertical closed-loop control.

7.2 Extended Kalman Filter (EKF) State Estimation

An Extended Kalman Filter from the ROS 2 `robot_localization` package fuses high-frequency IMU angular velocities and accelerations with depth odometry. Configured via `auv_sensors/config/ekf.yaml`, the filter publishes smooth 3D pose and twist estimates to `/odometry/filtered`, ensuring continuous navigation during brief camera occlusions.

7.3 Sim-to-Real Configuration Isolation

All hardware-specific parameters are isolated inside `auv_bringup/config/`. In simulation, `sim_params.yaml` connects to Gazebo bridge namespaces; for pool deployment, `real_params.yaml` maps to hardware drivers (V4L2 camera feeds, serial IMU, and PCA9685 PWM thruster boards). The algorithmic node implementations remain 100% identical.

8 Software Used

Software Component	Purpose in Our Stack
ROS 2 Humble	Core middleware – node lifecycle, DDS pub/sub messaging, launch system, parameter server.
Ultralytics YOLOv8n	Object detection model for gate, bins, QR codes, and flares; runs at 45 FPS onboard.
OpenCV (via CvBridge)	Image conversion (ROS ↔ NumPy), HSV thresholding, contour analysis, and QR decoding.
Gazebo Harmonic	Physics simulation + rendering. SDF world models for the pool, gate, bins, and mat.
gz-sim-thruster-system	Simulates physical propeller thrust on 8 thruster joints (BlueROV2 T200 equivalent).
stereo_image_proc	Stereo rectification and block-matching disparity map generation from left/right camera feeds.
robot_localization	Extended Kalman Filter fusing IMU and odometry into robust, filtered state estimates.
ros_gz_bridge	Bridges Gazebo topics (camera, odometry, cmd_thrust) into the ROS 2 topic graph.
Matplotlib / Plotly	Post-mission 2D/3D trajectory visualization with true-to-scale arena object footprints.
Python 3	All custom nodes are pure Python – rapid iteration without compilation overhead.

9 Sensor and Communication Circuit (Software Topology)

The diagrams below depict the flow of data across the vehicle. Raw optical streams from the front stereo pair and bottom camera pass through the bridge, splitting into YOLO/HSV detection paths and the disparity depth path. These merge at the localizer nodes to output 3D coordinates. High-level navigators consume these positions, issuing velocity commands to the closed-loop velocity controller, which commands the thruster allocator to actuate the 8 thruster joints.

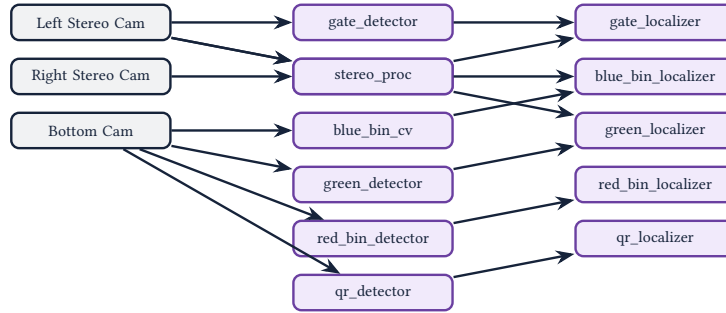


Fig 6. Perception Pipeline Topology: Optical Feeds to 3D Localizers

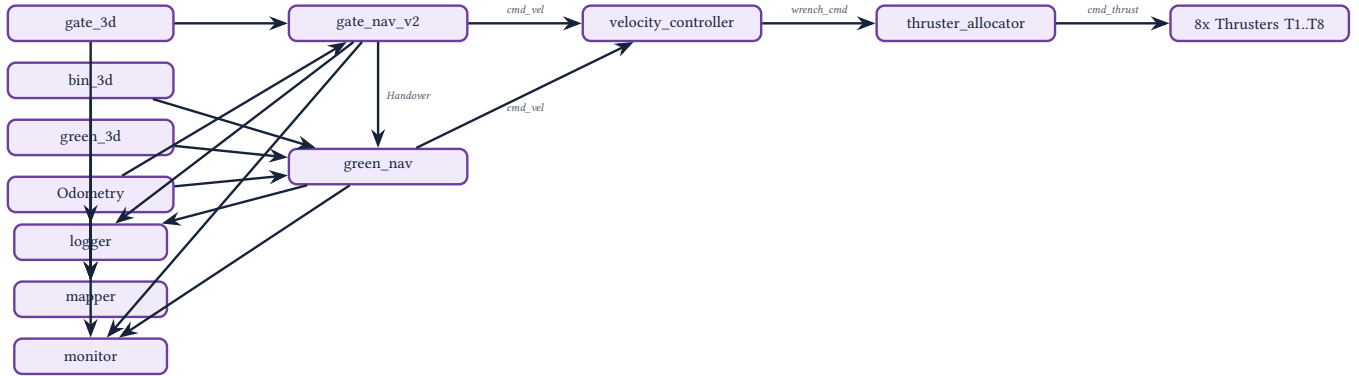


Fig 7. Planning, Closed-Loop Control, Thruster Allocation, and Telemetry Topology

9.1 Comprehensive ROS 2 Topic & Interface Catalog

The table below details every active topic across the 10-package software stack, outlining publishers, subscribers, message types, and functional roles:

Topic Name	Publisher Node	Message Type	Data Content & Role
.../stereo_front/left/image_raw	ros_gz_bridge	sensor_msgs/Image	Left stereo 800×600 optical feed for forward vision.
.../bottom/image_raw	ros_gz_bridge	sensor_msgs/Image	Downward camera stream for arena target inspection.
.../stereo_front/disparity	stereo_image_proc	stereo_msgs/DisparityImage	Dense disparity map for metric 3D depth computation.
/auv/gate_detection_2d	gate_detector_node	Float32MultiArray	Gate bounding box $[u, v, w, h, \text{conf}]$.
/auv/gate_position_3d	gate_localizer_node	Float32MultiArray	Gate 3D position $[x, y, z, \text{conf}]$ in vehicle body frame.
/auv/blue_bin_detection_bottom_2d	blue_bin_cv_detector	Float32MultiArray	2D coordinates of blue drum satisfying green mat overlap.
/auv/blue_bin_position_3d	blue_bin_localizer_node	Float32MultiArray	3D metric position of blue drum for visual centering.
/auv/green_position_3d	green_localizer_node	Float32MultiArray	3D position of green landing mat centroid.
/auv/qr_payload	qr_detector_node	std_msgs/String	Decoded text payload from floor QR target cubes.
/auv/mission_state	gate_navigator_v2_node	std_msgs/String	Active gate FSM state (FORWARD, TRACK, STOP, RETURN_CROSS, DONE).
/auv/green_mission_state	green_navigator_node	std_msgs/String	Post-gate FSM state (SEARCH_GREEN, CENTER...).
/auv/forward_reference_yaw	gate_navigator_v2_node	std_msgs/Float32	Locked reference heading (rad) for post-gate snap.
/model/auv_box/cmd_vel	gate_navigator_v2, green_navigator	geometry_msgs/Twist	Commanded body velocity: v_x (surge), v_y (sway), ω_z (yaw).
/auv/wrench_cmd	velocity_controller_node	geometry_msgs/Wrench	Net body wrench $[F_x, F_y, F_z, M_x, M_y, M_z]$.
/model/auv_box/joint/T_{1..8}/cmd_thrust	thruster_allocator_node	std_msgs/Float64	Newton thrust commands distributed across 8 thrusters.
/auv/odom	ros_gz_bridge	nav_msgs/Odometry	Ground-truth vehicle odometry and orientation.
/odometry/filtered	robot_localization (EKF)	nav_msgs/Odometry	Fused, drift-free state estimate from IMU + Odometry.
/auv/trail_map	trail_mapper_node	nav_msgs/Path	Full 3D mission trajectory history path.

10 Mission Telemetry & Mapping

During each mission, the AUV actively tracks its trajectory. The `trail_mapper_node` records the AUV's odometry at 1Hz, along with the precise 3D locations of the detected obstacles, generating a comprehensive top-down spatial footprint at shutdown. In parallel, `mission_logger_node` writes a 30+ column CSV capturing all vehicle dynamics, while `mission_monitor_node` provides a live ANSI terminal dashboard.

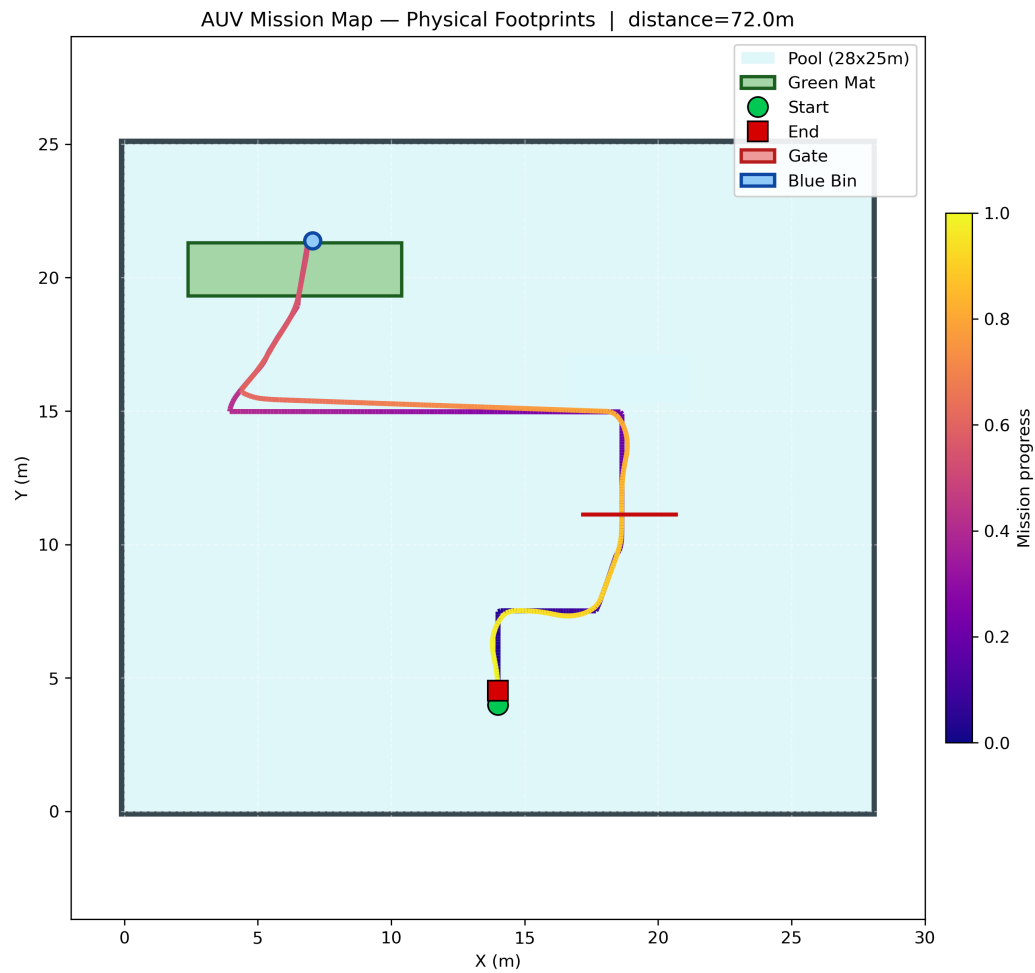


Fig 8. Auto-Generated 2D Mission Trajectory Map (distance = 72.0m) Showing the Gate Crossing, Lateral Traverse Along the Arena, and Approach to the Green Mat / Blue Bin.